CS 61A — Week 3

Solutions: Lab 2, Discussion 2 and actual exam practice

Lab 2 Q1: The Truth Will Prevail. **Idea.** Identify the last operand actually evaluated; record its value separately from its output effect.

expression	prompt display
`True and 13`	`13`
`False or 0`	`0`
`not 10`	`False`
`not None`	`True`
`True and 1/0`	`ZeroDivisionError` (ERROR)
`True or 1/0`	`True`
`-1 and 1 > 0`	`True`
`-1 or 5`	`-1`
`(1+1) and 1`	`1`
`print(3) or "`	`3`, then `"`
`0 or f(1)`	`'positive'`
`f(0) or f(-1)`	`'zero'`
`f(0) and f(-1)`	`"`

1. The true left operand of `True and 1/0` requires evaluating the division; the true left operand of `True or 1/0` skips it. A negative number is true, so `-1 and (1>0)` reaches the comparison.
2. `print(3)` prints 3 and returns false-valued None. The or expression therefore returns the empty string, which appears as `"` at the prompt.
3. `f(0)` returns nonempty `'zero'`, so or stops there but and continues to `f(-1)`, which returns the empty string.

Lab 2 Q2: Higher-Order Functions. **Idea.** Keep the returned pie function distinct from the string it returns when called.

input	display
`chocolate=cake()`	`beets`
`chocolate`	FUNCTION (pie)
`chocolate()`	`sweets`, then `'cake'`
`more_chocolate,more_cake=chocolate(),cake`	`sweets`
`more_chocolate`	`'cake'`
`snake(10,20)` before rebinding cake	FUNCTION (pie)
`snake(10,20)()` before rebinding cake	`sweets`, then `'cake'`
`snake(10,20)` after `cake='cake'`	`30`

1. Calling cake prints beets and returns pie; merely referring to chocolate does not call pie. Calling chocolate prints sweets and returns a string.
2. Simultaneous assignment computes both old right-hand values first. more_chocolate receives the string; more_cake receives the original cake function. No returned string is displayed by the assignment itself.
3. Initially cake and more_cake refer to the same function, so snake returns chocolate. After cake is rebound to a string they differ, so snake instead returns 10+20.

Lab 2 Q3: Lambda. **Idea.** A function is created before its body is evaluated; the body runs only after the corresponding call.

input	display
`lambda x:x`	FUNCTION
`a(5)`	5
`(lambda:3)()`	3
`c`	FUNCTION (zero-argument returned lambda)
`c()`	12
`d(square)`	16
`higher_order_lambda(g)(2)`	4
`call_thrice(lambda y:y+2)(5)`	11
`print_lambda`	FUNCTION
`one_thousand=print_lambda(1000)`	1000
`print(one_thousand)`	None

1. `b(8,4)` returns a zero-argument function retaining x = 8 and y = 4. Its later call computes 12. `d(square)` calls square on 4 and returns 16.
2. The higher-order lambda calls the supplied g on 2; call_thrice applies the increment-by 2 three times: $5 \to 7 \to 9 \to 11$.
3. print_lambda returns None after its output effect. The assignment stores None; printing that value displays the word None. The assignment and all other definitions produce no additional prompt display.

Lab 2 Q4: Piecewise. **Idea.** Wait for x before choosing which retained function to apply.

```
def piecewise(f, g, b):
    def h(x):
        return f(x) if x < b else g(x)
    return h
```

1. The function h retains access to f,g,b. A call to piecewise itself calls neither branch function.
2. With b = 0, negate and identity, h(6) chooses identity and gives 6; h (−1) chooses negate and gives 1. At x=b the g branch is selected.

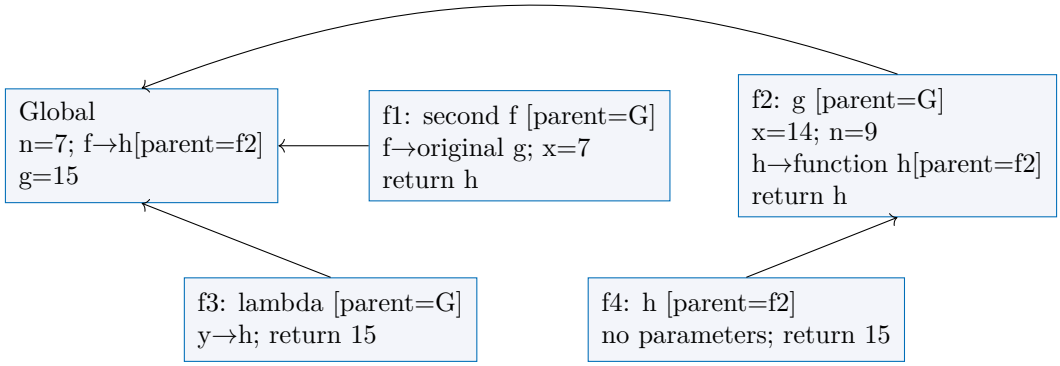
Lab 2 Q5: Count Cond. **Idea.** Retain the predicate, then start a new index and counter whenever the returned function receives n.

```
def count_cond(condition):
    def count(n):
        total, i = 0, 1
        while i <= n:
            if condition(n, i):
                total += 1
            i += 1
        return total
    return count
```

1. Before each test, total counts the successful indices 1 through $i - 1$. Testing condition(n,i) and then advancing i preserves this statement. At i=n+1 every permitted index has been tested.
2. `count_cond(lambda n,i:sum_digits(n*i)==5)` gives 1 at 10 (50), and 4 at 50 (50,500,1400,2300). Each listed product has digit sum 5.
3. `count_cond(lambda n,i:is_prime(i))` gives 1,2,2,3,8 at 2,3,4,5,20. The primes up to 20 are 2,3,5,7,11,13,17,19. Use the lab's supplied helpers; the positive-only primality helper is called only on i ≥ 1.

Lab 2 Q6: HOF Diagram Practice. **Idea.** The original f and the later f are different function values. Resolve the right-hand call before replacing its global name.

1. The second definition replaces the first global f. Evaluating `f(g,n)` calls that new f with its local parameter f bound to the original g function, and local x = 7. It reads global n = 7, then calls g(14).
2. g(14) has local x = 14,n = 9 and creates h with this call as parent. It returns h. The outer f also returns h, which becomes the new global f.
3. The global lambda receives h as y and calls y(). h finds x = 14 in its parent and returns 15. The lambda returns 15, then the assignment rebinds global g to 15. Global n stays 7.



Arrows are parent links, not call order. Global f and f2's h refer to the same returned function; original global g is replaced only after the lambda finishes.

The first f(x), with its unused local n = 8, is never called. Its name was replaced by the second definition; therefore it creates no call frame. The four actual user-defined calls are f, g, lambda, h.

Lab 2 Q7: Week 3 Survey. **Idea.** A completion token comes from the actual survey; it is not a programming exercise with a deducible answer.

1. The linked form says it no longer accepts responses. Its question text and completion token are unavailable here.
2. The starter's hash check checks a supplied token; it does not supply the token. No invented passphrase or personal responses are included.

Lab 2 Q8 (optional): Cycle. **Idea.** The counter tells both how many functions have run and which of the three must run next.

```
def cycle(f1, f2, f3):
    def g(n):
        def h(x):
            i = 0
            while i < n:
                if i % 3 == 0:
                    x = f1(x)
                elif i % 3 == 1:
                    x = f2(x)
                else:
                    x = f3(x)
                i += 1
            return x
        return h
    return g
```

1. f1,f2,f3 are retained by g; n is retained by h. On each call of h, i starts 0 and x is that call's input.
2. After i iterations, x is the result of the first i cyclic applications. Remainders 0,1,2 select f1,f2,f3; increasing i preserves the statement.
3. For input 2, add1/times2/add3 give $2 \to 3 \to 6 \to 9 \to 10$ through four steps. At six steps on 1, $1 \to 2 \to 4 \to 7 \to 8 \to 16 \to 19$. At $n = 0$ the loop skips all functions and returns the original input 5.

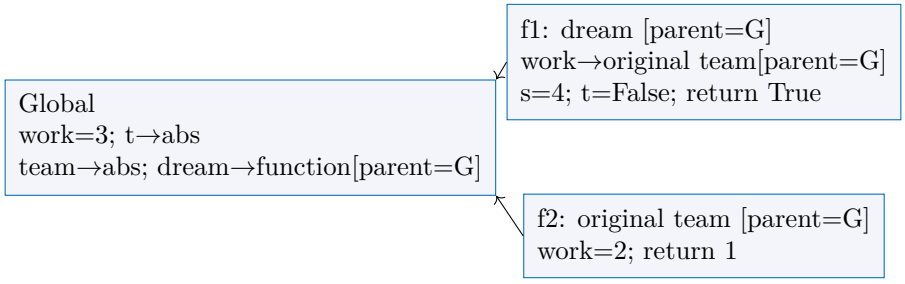
§1 Discussion 2

Q1: Warm Up. **Idea.** Use a separate local x for each lambda call.

1. The outer call binds $x = 5$. The inner call binds its own $x = 4$ but its body is the constant 3.
2. The outer body is therefore $2 \cdot 3 \cdot 5 = 30$. Assignment binds result 30 and produces no prompt output.

Q2: Teamwork. **Idea.** Determine the global t used by team before considering dream's separate local t.

1. The right side calls dream with work bound to the original team function and $s = 4$. Its condition calls that team on 2.
2. team(2) reads global t=abs and returns abs(2)−1 = 1. This is true, so dream assigns its own local t=not 4=False, then returns `not False`, which is `True`.
3. The final and therefore evaluates global t=abs and returns it. Only then does assignment change global team to abs; global work = 3 and t=abs remain unchanged.



Both call frames have Global as parent. The team call finds global abs, not the dream call's local t.

Q3: Make Keeper. **Idea.** Retain n, then scan its finite interval with the predicate supplied later.

```
def make_keeper(n):
    def f(cond):
        i = 1
        while i <= n:
            if cond(i):
                print(i)
            i += 1
    return f
```

1. Before each test all matching values below i have been printed once in increasing order. Test i, print it only if cond says true, and increase i.
2. At i=n+1 every permitted value has been considered. For n = 5 an even predicate prints 2,4; true prints 1,2,3,4,5; false prints nothing. With no explicit return, each returned-function call returns None.

Q4: Digit Finder. **Idea.** Keep the requested position k while waiting for the integer x.

```
def find_digit(k):
    assert k > 0
    return lambda x: (x // 10**(k - 1)) % 10
```

1. For k = 2,x = 3456, division by 10 leaves 345; remainder 10 gives 5. For 5678 it leaves 567 and gives 7.
2. At k = 1,x = 10, no digits are removed and remainder 10 is 0. For k = 4,x = 789, division by 1000 is 0, so the absent digit gives 0. The displayed implementation uses no string or indexing operation.

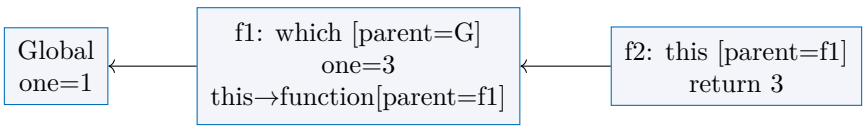
Q5: Match Maker. **Idea.** One unequal existing pair disproves the claim; success requires all such pairs to pass.

```
def match_k(k):
    def check(x):
        while x // (10**k) > 0:
            if x % 10 != (x // (10**k)) % 10:
                return False
            x //= 10
        return True
    return check
```

1. Compare the last digit with the last digit after stripping k places. After stripping one more digit from x, the next adjacent position is ready; no pair is omitted.
2. For 1010,k = 2, compare 0 with 0, then 1 with 1. For 2010,k = 2, the second comparison is 1 with 2, so return false. For 123123,k = 3 all three corresponding pairs agree; at k = 2 the first pair is 3 versus 1 and fails.
3. When x has at most k digits the condition is false, since no full pair remains. Thus the single digit 1 at k = 1 returns true. For 2111111111111111 the scan eventually reaches 21 and detects 1 versus 2.

Q6 (optional): Which One. **Idea.** The first return in each call makes the later statements unreachable.

1. which binds local one = 3 and creates this with that frame as parent. It returns this before the assignment one = 4.
2. The returned this finds one = 3 in that parent and returns it before its second return. Thus `which()()` displays 3; global one remains 1.



The first matching binding on the parent chain is local one = 3. Global one = 1 is not used.

Q7 (optional): Choose Wisely. **Idea.** Reject the first failing digit; if there is no failure before the end, every digit passed.

```
def every(t):
    def digit(n):
        assert n > 0
        while n:
            if not t(n % 10):
                return False
            n = n // 10
        return True
    return digit
```

1. The blanks are `not t(n%10)`, `return False`, `True`, and `digit`. Before each test all removed digits passed t.
2. For 37511 the odd-digit tests on 1,1,5,7,3 all pass. For 2023, the first 3 passes but the next 2 fails, so the result is false.

Q8 (optional): Nice Dice. **Idea.** Count passing digits once, then supply the threshold as a new predicate.

```
def count_if(f):
    def g(n):
        count = 0
        while n:
            if f(n % 10):
                count += 1
            n = n // 10
        return count
    return g

def count_at_least(k):
    return count_if(lambda d: d >= k)
```

1. The first blank is `f(n%10)`. Before each test, count is the number of removed digits passing f. Testing the last digit, incrementing if necessary and removing it preserves the statement.
2. The second blank is `lambda d:d>=k`. It retains k and lets count_if perform the entire scan; no second loop is needed.
3. In 431663334231 there are five 3s, so the equality test counts 5. The only digits below 3 are 1,2,1; the other 9 meet threshold 3.

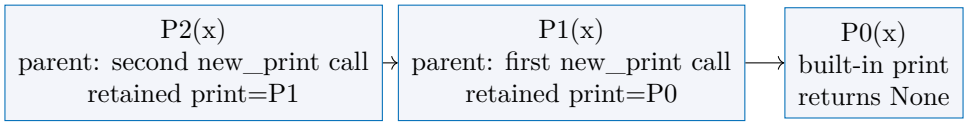
§2 Actual exam practice

Fall 2022 MT 1Q4(a): Funsquare. **Idea.** Check the outer interface and the deferred inner computation separately.

1. D and F are correct: each accepts only f and returns a one-argument function whose later body computes f(f(x)). For triple at 5 the inner call gives 15, then the outer gives 45.
2. A immediately calls f on the function f, which is not the requested later input. B returns a zero-argument function, which cannot accept x and also has the wrong body f(f).
3. C and E each require an additional outer x, so funsquare(triple) raises TypeError. C's inner computation would be suitable after two outer arguments, but that is the wrong required signature; E directly returns a result rather than the required function.

Fall 2026 MT 1Q2: New Print. **Idea.** Name each wrapper separately; its parameter named print still refers to the previous wrapper after the global name is rebound.

1. Let P0 be built-in print. The first returned f is P1, whose parent retains print=P0. The second returned f is P2, whose different parent retains print=P1. Global og=P0, print=P2.
2. Calling P2(2) first calls P1(2). P1 calls P0(2), printing 2 and receiving None, so P1 returns 2. P2 therefore has value 2 and enters its if.
3. P2 returns P1('What?'). That calls P0('What?'), printing What? and returning None, so P1 returns the string 'What?'. Finally og prints the words print returned followed by that returned string.



Each wrapper calls the print stored in its own parent. Rebinding the global name does not make P1 call P2.

The complete output is:

```
2
What?
print returned What?
```

Fall 2026 MT 1Q4.3: Make Gap (optional A+). **Idea.** Gap needs a consecutive index 1,2,.... Count how many indices pass use, then translate its jth index to the jth passing original index.

```
def f(use, go, then):
    y, i = 0, 0
    while go(y, i):
        i += 1
        if use(i):
            y += 1
    return then(y, i)

def make_gap(use):
    return lambda n, term: gap(
        f(use, lambda y, i: i < n, lambda y, i: y),
        lambda j: term(f(use, lambda y, i: y < j,
                         lambda y, i: i)))
```

1. In the supplied f, i is the number of indices tested and y the number passing use. For `go=lambda y,i:i<n`, the loop tests exactly 1 through n; `then=lambda y,i:y` returns their passing count m.
2. For any $1 \le j \le m$, `go=lambda y,i:y<j` stops at the jth passing index. Returning i locates that index. Applying term there supplies the jth value of the selected sequence to gap.
3. For a stable predicate use, gap therefore compares exactly consecutive selected values. With odd indices through 8, the first f returns 4; the inner f calls locate 1,3,5,7, whose mod 5 values are 1,3,0,2. Differences 2,3,2 have maximum 3.
4. At least two indices pass, so gap's sequence has at least two terms and every requested jth index exists. The answer uses only the permitted lambdas and calls. The if inside supplied f is provided code, not a forbidden if introduced in the answer.